# Guía del Proyecto

*Cómo instalar, correr y colaborar en el proyecto (versión 2)*

Guía para que cualquier integrante del equipo pueda clonar el proyecto, reproducir los resultados y trabajar con ramas y pull requests.

## 1. Requisitos

- Git
- Python 3.12 o superior (el proyecto usa 3.14)
- Docker Desktop (para levantar la API y el frontend)
- VS Code (recomendado)

## 2. Clonar e instalar

```
git clone https://github.com/cristian-aguilera-ing/proyecto-ds-ypf.git
cd proyecto-ds-ypf
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pip install -e .                   # instala el paquete ypf_ds (carpeta src/)
code .
```

Cuando el entorno está activo, la terminal muestra (.venv) al principio de la línea.

## 3. Datos

Los CSV no se versionan (pesan 139 MB y 34 MB). Hay que descargarlos y guardarlos en data/raw/ sin renombrar:

- produccin-de-pozos-de-gas-y-petrleo-no-convencional.csv (obligatorio; link en el README)
- capitulo-iv-pozos.csv (opcional: si no está, el pipeline funciona igual)

El dataset ya limpio (data/processed/*.parquet) y los modelos entrenados (models/) sí están en el repositorio: con eso alcanza para correr la API y los notebooks 03 a 05 sin descargar nada.

## 4. Correr el pipeline

| Comando | Qué hace | Tiempo aprox. |
|---|---|---|
| python -m ypf_ds.data.clean | Limpieza → data/processed/ | 2 s |
| python -m ypf_ds.models.train | Entrena los 4 modelos y el clustering → models/ | 25 min |
| pytest -q | Corre los 15 tests | 2 s |
| uvicorn api.app:app --reload | API en http://localhost:8000/docs | — |
| make notebooks | Ejecuta todos los notebooks | 5 min |
| mlflow ui --backend-store-uri sqlite:///mlflow.db | Ver los experimentos registrados | — |

## 5. Levantar todo con Docker

```
docker compose up --build -d      # construye y levanta API + frontend
docker compose ps                  # estado de los contenedores
docker compose down                # apagar
```

- Frontend: http://localhost:8080
- API y documentación interactiva (Swagger): http://localhost:8000/docs

Reentrenar dentro de un contenedor:

```
docker compose --profile train run --rm train
```

Cómo leer los resultados del frontend: docs/guia_frontend.md.

## 6. Estructura del código

| Archivo | Responsabilidad |
|---|---|
| src/ypf_ds/config.py | Rutas, umbrales de limpieza y cortes temporales: todo lo configurable está acá |
| src/ypf_ds/data/load.py | Lectura lazy del CSV con Polars y diagnóstico de calidad |
| src/ypf_ds/data/clean.py | Limpieza, Capítulo IV, grilla mensual y tabla de pozos |
| src/ypf_ds/features/build.py | Features y targets de los modos A y B |
| src/ypf_ds/models/train.py | Optuna, entrenamiento, evaluación, selección de variables y metadata.json |
| src/ypf_ds/models/predict.py | Lógica de predicción compartida por la API |
| src/ypf_ds/models/baselines.py | Persistencia, Arps y curva mediana |
| src/ypf_ds/models/cluster.py | SVD + K-Means y perfiles de cluster |
| api/app.py · model.py · schemas.py | Endpoints, carga única de modelos y validación de entradas |
| frontend/ | index.html, app.js y estilos.css |

## 7. Flujo de trabajo con Git

Nunca se trabaja directo sobre main: cada tarea va en su propia rama y entra a main por pull request, con revisión de otro integrante.

1. Actualizar main antes de empezar.
2. Crear una rama con un nombre descriptivo.
3. Hacer commits chicos con mensajes claros.
4. Subir la rama y abrir el pull request en GitHub.
5. Otro integrante revisa y aprueba; después se hace el merge.

```
git checkout main
git pull origin main
git checkout -b feature/mi-tarea
git add archivo1 archivo2
git commit -m "Agrego validación de profundidad en clean.py"
git push origin feature/mi-tarea
```

Antes de abrir el pull request: correr pytest -q y verificar que no se subieron archivos pesados (data/raw está en .gitignore).

## 8. Problemas frecuentes

| Problema | Solución |
|---|---|
| ModuleNotFoundError: ypf_ds | Activar el entorno y correr pip install -e . |
| FileNotFoundError en data/raw | Descargar el CSV principal (ver sección 3) |
| El frontend dice “API no disponible” | docker compose ps: la API tiene que figurar como healthy |
| Docker no responde | Abrir Docker Desktop y esperar a que inicie |
| Los notebooks tardan mucho | Correr primero la limpieza; los notebooks usan los Parquet de data/processed/ |